SR UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

LAB ASSIGNMENT :- 3

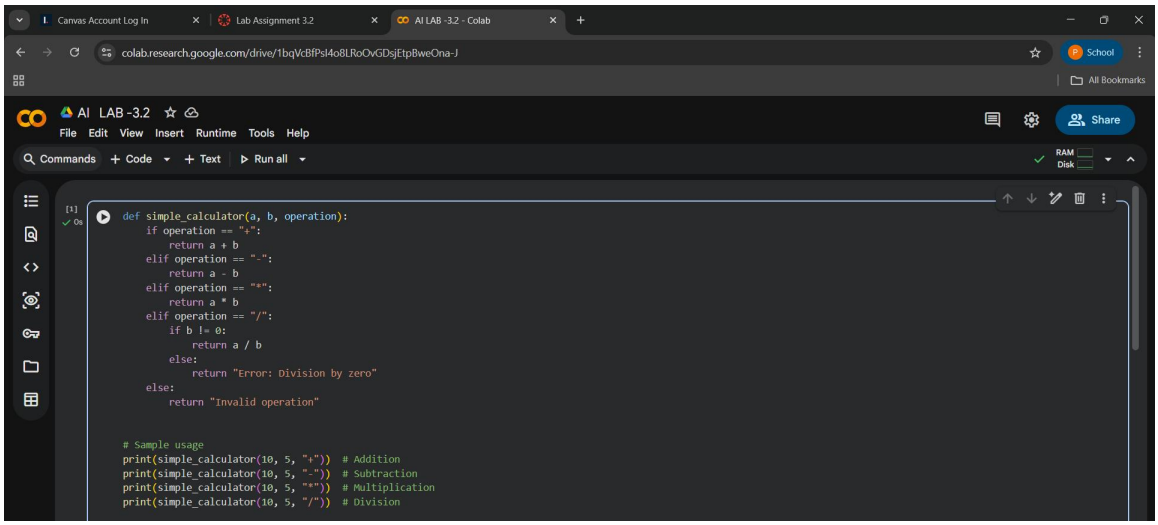
Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name:- Dr. Rishabh Mittal	

STUDENT NAME	PENTAPURIYESHWANTHREDDY
ENROLLMENT NUMBER	2403A51L39
SUBJECT NAME	AI Assisted Coding
COURSE CODE	23CS002PC304
YEAR/SEM	III/II
BATCH	52

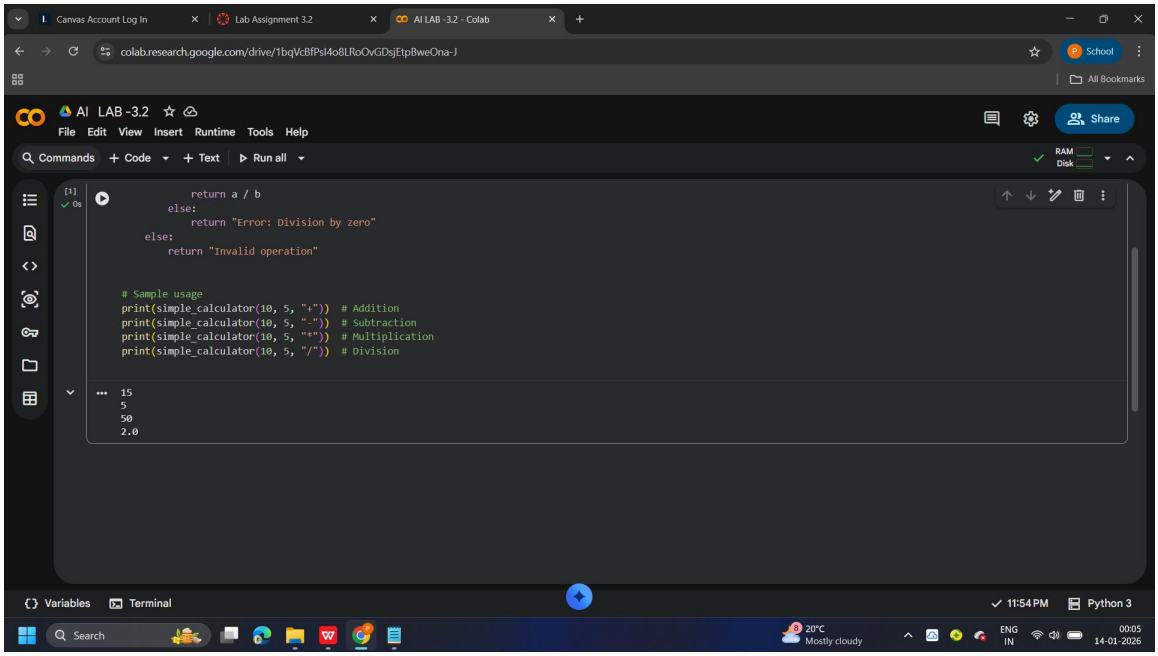
TASK:-1 Progressive Prompting for Calculator Design: Ask the AI to design a simple calculator program by initially providing only the function name. Gradually enhance the prompt by adding comments and usage examples.

PROMPT :- ou are an AI programming assistant. Design a simple calculator program in Python. Initially, provide **only the function name** (no implementation). Then, step by step, complete the function to perform basic arithmetic operations (addition, subtraction, multiplication, and division). Finally, demonstrate the function with sample inputs and outputs.

CODE:-



OUTPUT:-



CODE Explanation

1. Function Purpose

The function simple_calculator(a, b, operation) is designed to perform a basic arithmetic operation on two numbers (a and b) based on the operator provided.

2.**Inputs to the Function**

`a` → first number

`b` → second number

`operation` → a string (`"+"`, `"-"`, `"*"`, `"/"`) that decides which calculation to perform.

3 Operation Selection Using Conditions

The function uses if and elif statements to compare the operation value and choose the correct arithmetic logic.

4. Addition Logic (+)
If operation is "+", the function adds a and b and immediately returns the result.

5. **Subtraction and Multiplication Logic (-, *)**

- "-" subtracts b from a.
- "*" multiplies a and b.
  Each operation returns its result directly
- "*" multiplies a and b.
  Each operation returns its result directly..

6. **Division Logic with Safety Check (/)**
Before dividing, the function checks if b is not zero.

If b ≠ 0, it performs division.

If b = 0, it avoids an error and returns a meaningful message.

KEY POINT Overall Understanding:
This function uses conditional logic, safe error handling, and early returns to perform accurate calculations while preventing runtime errors.

TASK:-2 Refining Prompts for Sorting Logic: Start with a vague prompt for sorting student marks,
then refine it to clearly specify sorting order and constraints.

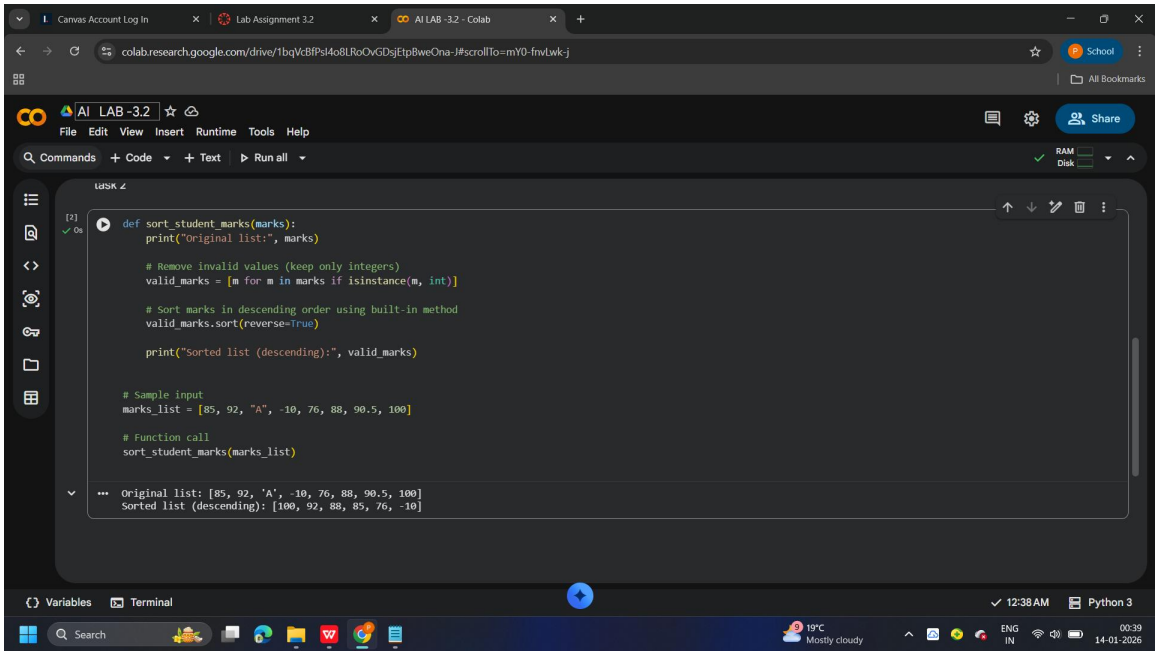
Task 2 Prompt :-

Create a program to sort student marks.

Prompt

Generate a Python function that accepts a list of student marks (integers only), removes any invalid values, and sorts the marks in descending order. The function should display both the original list and the sorted list, and use an efficient built-in sorting method..

CODE & OUTPUT



CODE explanation

Explanation

1.**Input:** A list containing student marks (may include invalid values).

2.**Validation:** Keeps only integer values using `isinstance(m, int)`.

3.**:** Uses Python's efficient built-in `.sort()` method with `reverse=True`.

4. **Output:** Displays both the original list and the cleaned, sorted list.

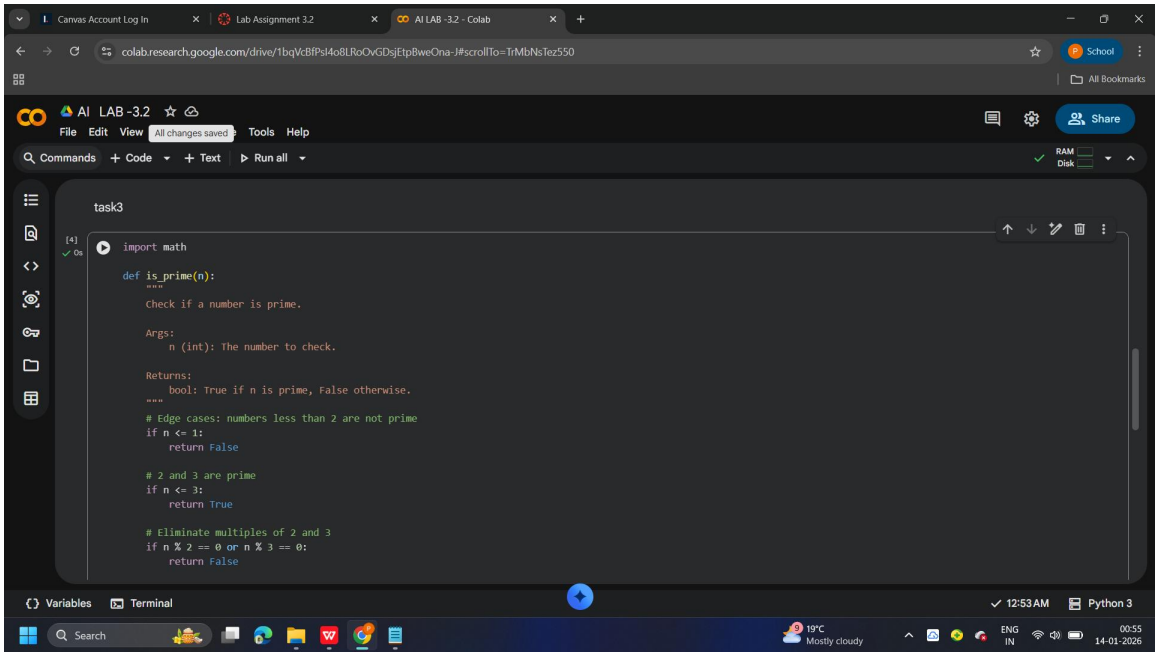
TASK:-3 Task Description-3

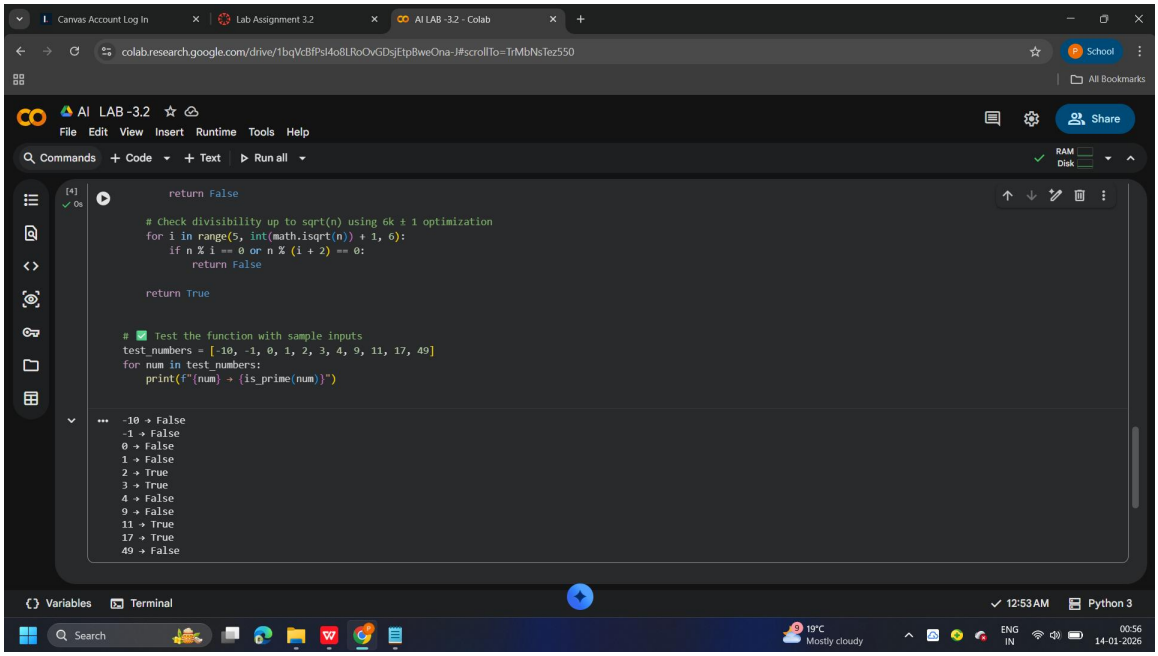
- Few-Shot Prompting for Prime Number Validation: Provide multiple input-output.

examples for a function that checks whether a number is prime.
Observe how few-shot.
prompting improves correctness.
Expected Output-3
• Improved prime-checking function with better edge-case handling.

Task 3 prompt 3:-You are a senior Python engineer and algorithm specialist. Your task is to generate a **correct, efficient, and production-ready** Python function named is_prime(n) that determines whether an integer is a prime number

CODE & OUTPUT


Canvas Account Log In
Lab Assignment 3.2
AI LAB -3.2 - Colab
colab.research.google.com/drive/1bqVcBfPsI4o8LRoOvGDsjEtpBweOna-J#scrollTo=TrMbNsTez550
AI LAB -3.2
File Edit View Tools Help
All changes saved
Commands + Code + Text Run all
Share
task3
[4]
import math
def is_prime(n):
"""
Check if a number is prime.
Args:
n (int): The number to check.
Returns:
bool: True if n is prime, False otherwise.
"""
# Edge cases: numbers less than 2 are not prime
if n <= 1:
return False
# 2 and 3 are prime
if n <= 3:
return True
# Eliminate multiples of 2 and 3
if n % 2 == 0 or n % 3 == 0:
return False
Variables Terminal
12:53 AM Python 3



CODE Explanation

Prime Number Function - Logic Explanation (6 Key Points)

1.Handle invalid inputs

Any number less than or equal to 1 is not prime, so the function immediately returns `False`.

2.Directly accept smallest primes

Numbers 2 and 3 are prime by definition, so the function returns `True` without further checks.

3. **Remove obvious non-primes early**

If the number is divisible by 2 or 3, it cannot be prime and the function returns `False`.

4. **Limit divisor checking to √n**

   A number only needs to be checked for factors up to its square root, improving efficiency.

5. **Apply 6k ± 1 optimization**

   All primes greater than 3 follow the pattern `6k - 1` or `6k + 1`, so the loop checks only these values.

6. **Final decision**

   If no divisor is found within the loop, the number has exactly two divisors and is declared prime (`True`).

TASK:-4 Task Description-4

Prompt-Guided UI Design for Student Grading System: Create a user interface for a

student grading system that calculates total marks, percentage, and grade based on user

input.

Expected Output-4

• Well-structured UI code with accurate calculations and clear output display.

PROMPT

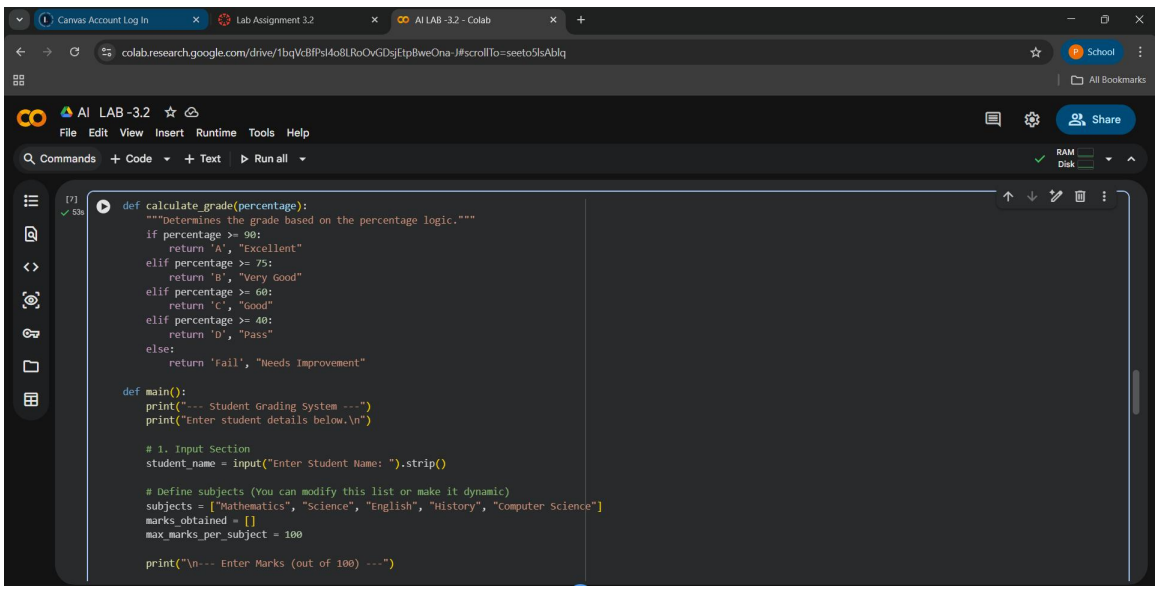
Act as an experienced front-end developer and UI designer. Design a clean and user-friendly **Student Grading System user interface** that allows users to enter student details and marks.

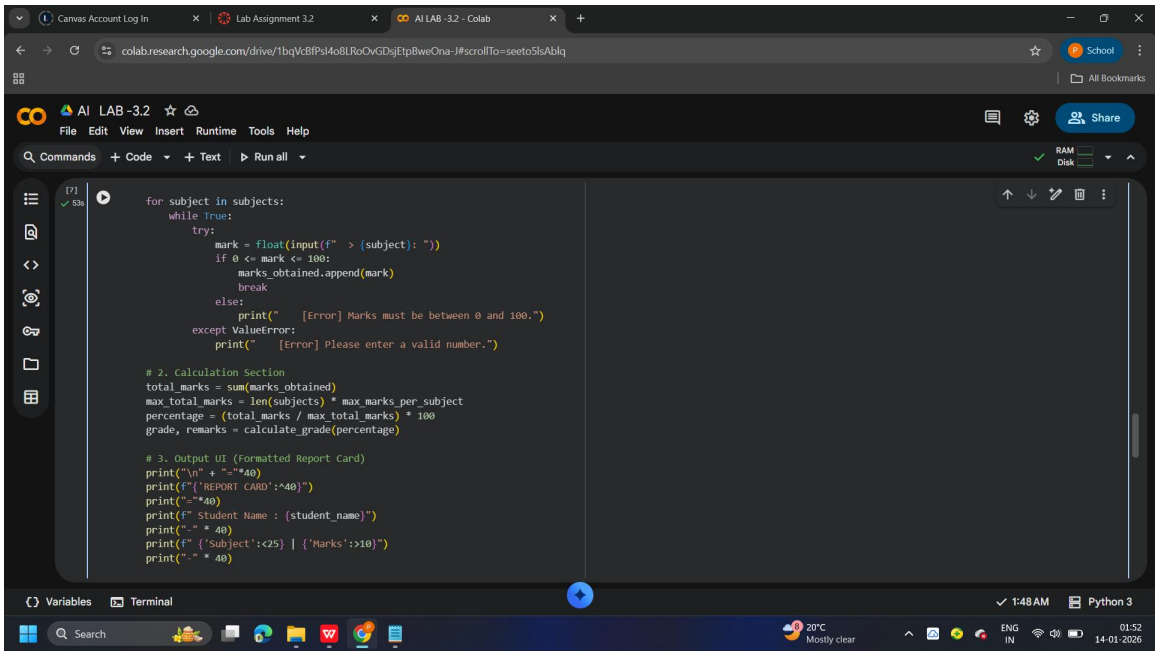
Functional Requirements:
• Accept user input for student name and marks of multiple subjects
• Calculate **total marks** automatically
• Calculate **percentage** based on total marks
• Assign **grade** using the following logic:
  – Percentage ≥ 90 → Grade A
  – Percentage ≥ 75 → Grade B

– Percentage ≥ 60 → Grade C
– Percentage ≥ 40 → Grade D
– Percentage < 40 → Fail

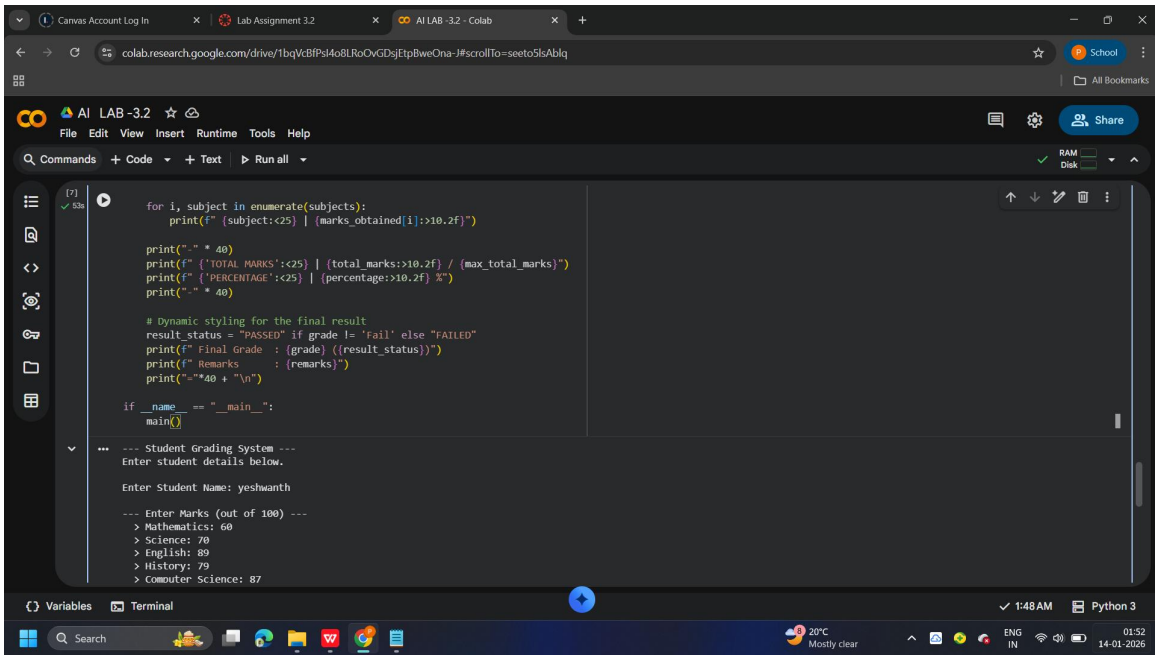
Output:
• Clear display of results after user input

CODE





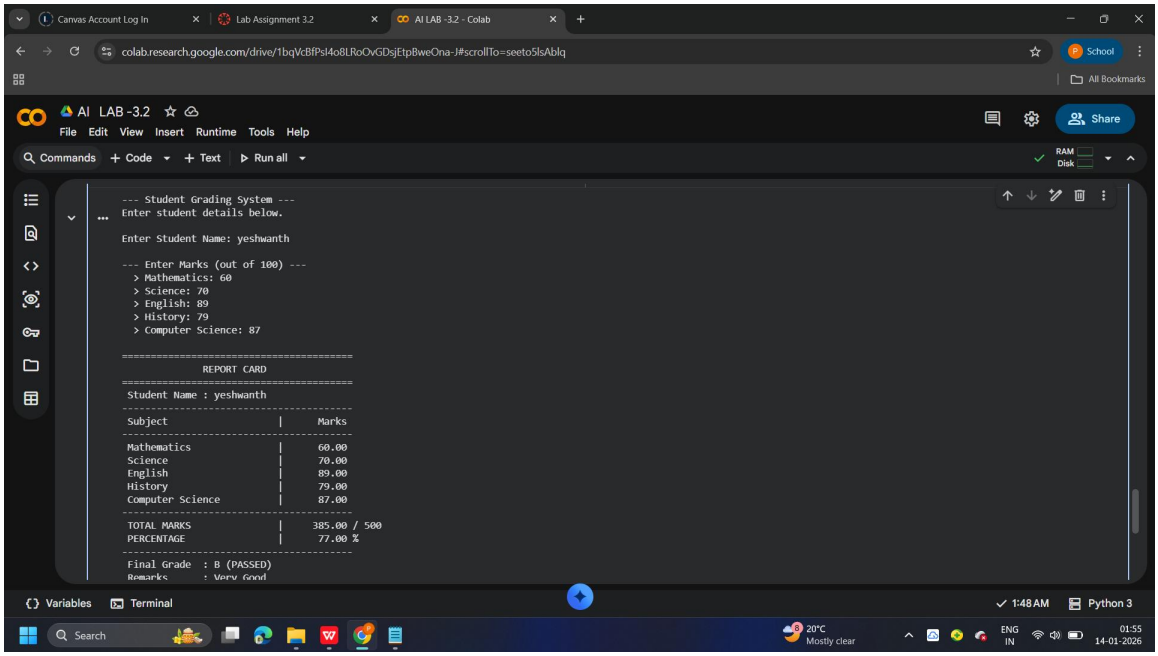
```python
    for subject in subjects:
        while True:
            try:
                mark = float(input(f"  > {subject}: "))
                if 0 <= mark <= 100:
                    marks_obtained.append(mark)
                    break
                else:
                    print("    [Error] Marks must be between 0 and 100.")
            except ValueError:
                print("    [Error] Please enter a valid number.")

    # 2. Calculation Section
    total_marks = sum(marks_obtained)
    max_total_marks = len(subjects) * max_marks_per_subject
    percentage = (total_marks / max_total_marks) * 100
    grade, remarks = calculate_grade(percentage)

    # 3. Output UI (Formatted Report Card)
    print("\n" + "="*40)
    print(f"{'REPORT CARD':^40}")
    print("="*40)
    print(f" Student Name : {student_name}")
    print("-" * 40)
    print(f" {'Subject':<25} | {'Marks':>10}")
    print("-" * 40)
```



```python
    for i, subject in enumerate(subjects):
        print(f" {subject:<25} | {marks_obtained[i]:>10.2f}")

    print("-" * 40)
    print(f" {'TOTAL MARKS':<25} | {total_marks:>10.2f} / {max_total_marks}")
    print(f" {'PERCENTAGE':<25} | {percentage:>10.2f} %")
    print("-" * 40)

    # Dynamic styling for the final result
    result_status = "PASSED" if grade != 'Fail' else "FAILED"
    print(f" Final Grade  : {grade} ({result_status})")
    print(f" Remarks      : {remarks}")
    print("="*40 + "\n")

if __name__ == "__main__":
    main()
```

```
--- Student Grading System ---
Enter student details below.

Enter Student Name: yeshwanth

--- Enter Marks (out of 100) ---
  > Mathematics: 60
  > Science: 70
  > English: 89
  > History: 79
  > Computer Science: 87
```

OUTPUT:-


Canvas Account Log In
Lab Assignment 3.2
AI LAB -3.2 - Colab
colab.research.google.com/drive/1bqVcBfPsI4o8LRoOvGDsjEtpBweOna-J#scrollTo=seeto5lsAblq
AI LAB -3.2
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Share
--- Student Grading System ---
Enter student details below.
Enter Student Name: yeshwanth
--- Enter Marks (out of 100) ---
> Mathematics: 60
> Science: 70
> English: 89
> History: 79
> Computer Science: 87
REPORT CARD
Student Name : yeshwanth
Subject | Marks
Mathematics | 60.00
Science | 70.00
English | 89.00
History | 79.00
Computer Science | 87.00
TOTAL MARKS | 385.00 / 500
PERCENTAGE | 77.00 %
Final Grade : B (PASSED)
Variables
Terminal
1:48 AM
Python 3

Here is the logic in **5 simple steps**:

> **1.Grade Function:** A small helper block that decides if a score is 'A', 'B', etc., based on the percentage rules.
>
> **2.Input Loop:** The code cycles through the list of subjects (Math, Science, etc.) and asks the user to enter marks for each.
>
> **3.Safety Check:** It uses a `try/except` block to stop the program from crashing if you type text instead of numbers.
>
> **4.Math:** It sums up your marks and divides by the total possible score to get the percentage.
>
> **5.Display:** It uses **f-strings** (formatting tools) to align the text and numbers into a neat, readable table.

TASK:-5 Task Description-5

Analyzing Prompt Specificity in Unit Conversion Functions: Improving a Unit Conversion Function (Kilometers to Miles and Miles to Kilometers) Using Clear

Instructions.

Expected Output-5
• Analysis of code quality and accuracy differences across multiple prompt variations.

PROMPT :-

Act as a Python developer. Create a well-structured Python function that converts values between **kilometers and miles**.

`Requirements:`

Accept two inputs:

A numeric value

A unit type (`"km"` or `"miles"`)

Perform the correct conversion:

Kilometers to miles

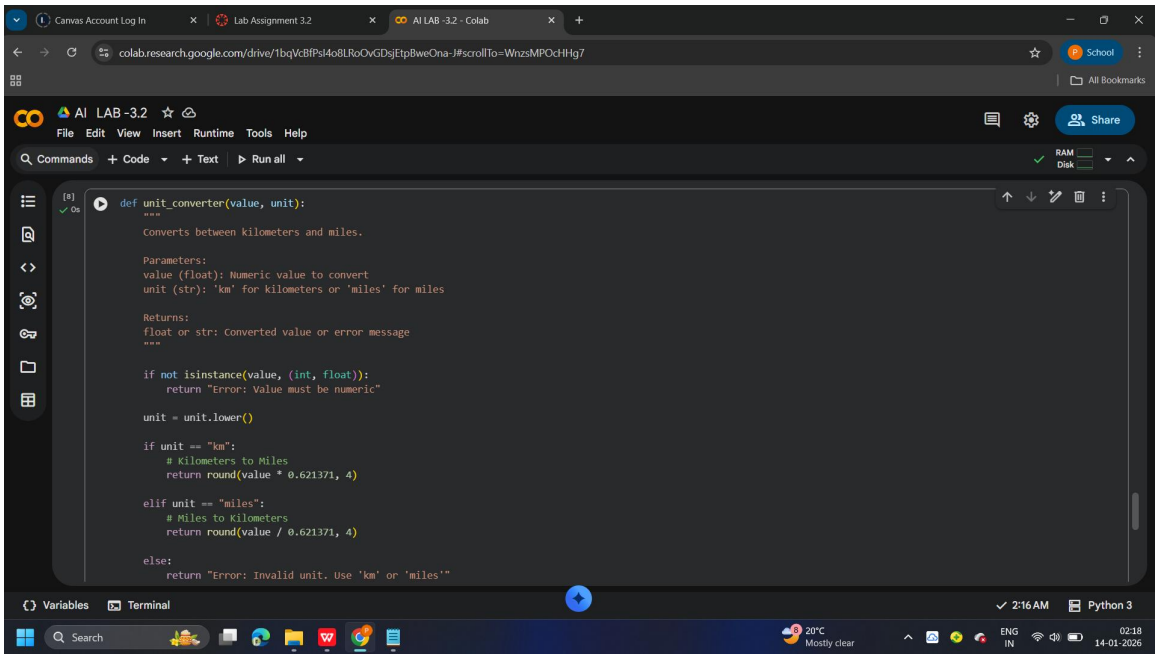
Miles to kilometers

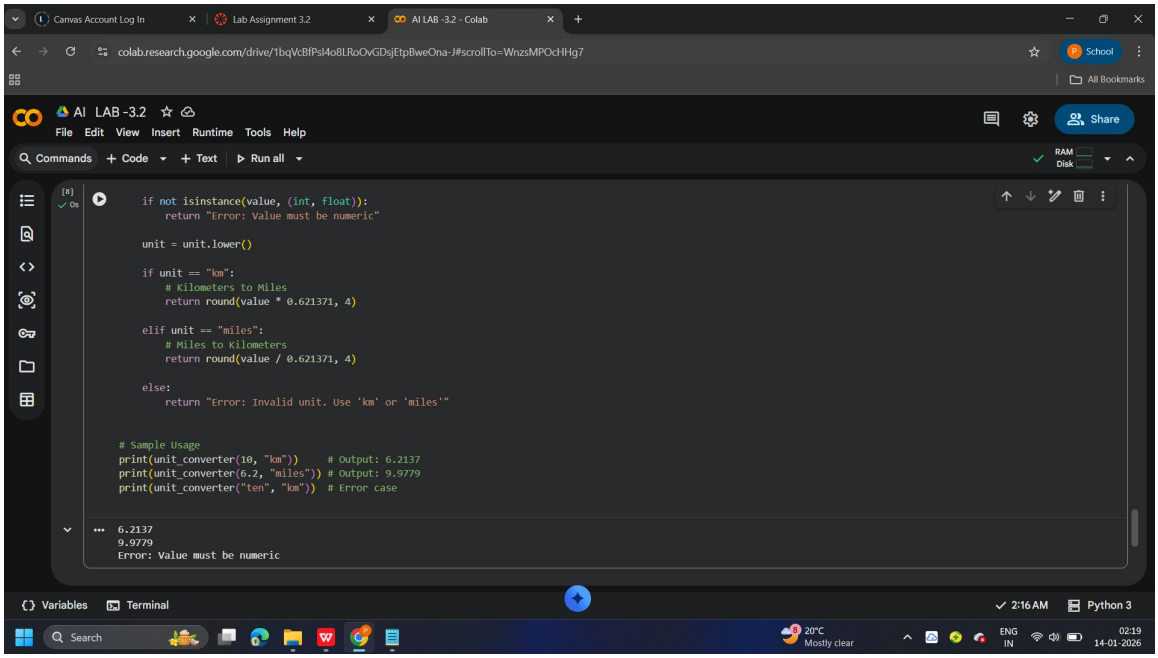
Use accurate conversion logic

Return the converted value with appropriate precision

Include basic error handling for invalid unit inputs

CODE & OUTPUT


Canvas Account Log In
Lab Assignment 3.2
AI LAB -3.2 - Colab
colab.research.google.com/drive/1bqVcBfPsI4o8LRoOvGDsjEtpBweOna-J#scrollTo=WnzsMPOcHHg7
AI LAB -3.2
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Share
[8]
def unit_converter(value, unit):
"""
Converts between kilometers and miles.
Parameters:
value (float): Numeric value to convert
unit (str): 'km' for kilometers or 'miles' for miles
Returns:
float or str: Converted value or error message
"""
if not isinstance(value, (int, float)):
return "Error: Value must be numeric"
unit = unit.lower()
if unit == "km":
# Kilometers to Miles
return round(value * 0.621371, 4)
elif unit == "miles":
# Miles to Kilometers
return round(value / 0.621371, 4)
else:
return "Error: Invalid unit. Use 'km' or 'miles'"
Variables
Terminal
2:16 AM
Python 3
Search
20°C
Mostly clear
ENG
IN
02:18
14-01-2026



CODE Explanation

1.Function Definition & Purpose

The function unit_converter(value, unit) is designed to convert distance values between kilometers and miles. It accepts a numeric value and a unit type, then returns the correctly converted result.

2.Input Validation

The function first checks whether the provided value is a number (int or float). This prevents runtime errors and ensures only valid numeric inputs are processed.

3.Unit Normalization

The unit input is converted to lowercase using unit.lower() so that inputs like "KM", "Km", or "miles" are treated consistently.

4.Conversion Logic

If the unit is "km", the value is multiplied by 0.621371 to convert kilometers to miles.

If the unit is "miles", the value is divided by 0.621371 to convert miles to kilometers.

The result is rounded to 4 decimal places for accuracy and readability.

5.Error Handling & Output

If an unsupported unit is entered, the function returns a clear error message. This ensures the function behaves safely and provides meaningful feedback instead of failing unexpectedly.